GUÍA TÉCNICA · EQUIPO .NET

RoslynGate

Cómo un agente de IA entiende .NET con Roslyn, sin convertir el contexto en una sopa de texto.

Arquitectura, pipeline de compilación, resolución semántica de referencias, contrato MCP, rendimiento medido y límites reales. Este documento explica dónde se conecta RoslynGate con MSBuild y Roslyn, qué hace la IA con cada respuesta y qué se puede escalar después.

CONTENIDO

Qué encontrarás aquí

01	El problema: texto no es semántica	03
02	Dónde entra RoslynGate en el pipeline .NET	04
03	El viaje de una consulta	05
04	Cómo se conoce una referencia	06
05	Arquitectura interna	07
06	Cómo lo usa un agente de IA	08
07	Rendimiento y economía de tokens	09
08	Límites, frescura y SDK	10
09	Escalabilidad y expansión	11

Resumen en una frase

RoslynGate convierte preguntas sobre símbolos .NET en consultas semánticas contra una solución cargada por MSBuild, y devuelve JSON compacto con archivo y línea para que un agente decida sin leer medio repositorio.

4–11 ms	**99**	**21–34%**	**~27 s**
consultas en caliente	proyectos en la prueba legacy	menos tokens en A/B real	carga inicial del workspace

> Las cifras de tokens corresponden a dos consultas medidas con Claude Code sobre una solución real; no son un porcentaje universal. La latencia es el precio de cargar MSBuild una vez por proceso.

01 — EL PROBLEMA

Texto no es semántica

Un agente que busca símbolos con `grep` encuentra letras. Un compilador encuentra significado. La diferencia importa cuando hay overloads, partial classes, genéricos, código generado o proyectos que se referencian entre sí.

1 Falsos positivos

La palabra “Doors” aparece en comentarios, strings, documentación y nombres de otros tipos. Una búsqueda textual los mezcla con referencias reales.

2 Falsos negativos

Las referencias semánticas pueden vivir en archivos generados o resolverse a través de una interfaz. El texto no siempre las conecta.

3 Overloads

`Open()`, `Open(string)` y `Open(int)` comparten nombre pero no símbolo. El agente necesita desambiguar.

4 Contexto caro

Volcar archivos enteros consume tokens y desplaza información útil. El objetivo es devolver la ubicación correcta, no el código completo.

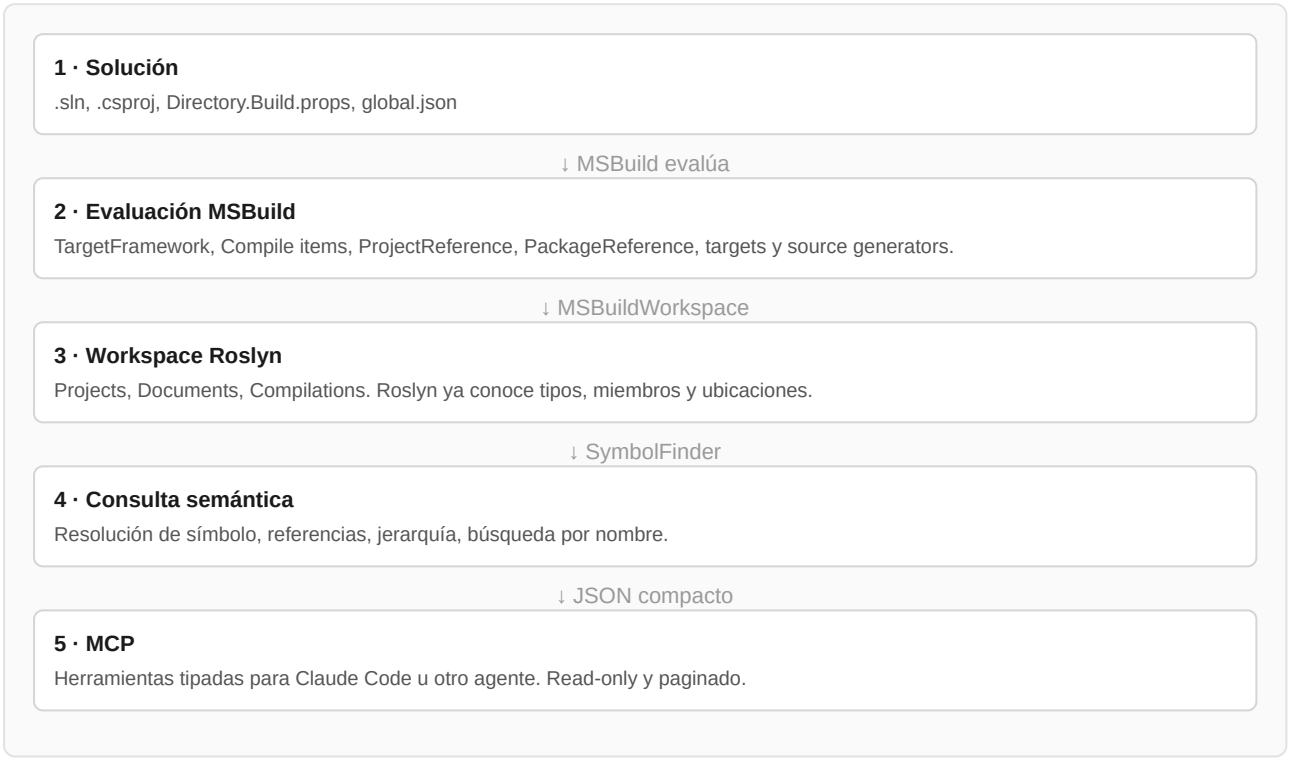
> *RoslynGate no reemplaza a grep para texto libre. Lo complementa cuando la pregunta es “qué símbolo es este y quién lo usa”.*

La herramienta es read-only: observa la solución, no edita código ni ejecuta tests.

02 — DÓNDE ENTRA

El pipeline .NET, etapa por etapa

RoslynGate no se inyecta dentro del compilador. Se coloca al lado: usa MSBuild para preparar la misma información que usaría un build, y después consulta esa información con Roslyn.



Qué aporta cada etapa

ETAPA	RESPONSABLE	QUÉ RESUELVE
Evaluación	MSBuild / SDK	Qué archivos y referencias pertenecen realmente al proyecto.
Compilación	Roslyn	Tipos, miembros, overloads, scopes y semantic model.
Consulta	RoslynGate	Orden, deduplicación, paginación y formato para el agente.

03 — UNA CONSULTA

El viaje de una pregunta

El agente no ve una solución .NET. Ve herramientas MCP. RoslynGate traduce entre ambos mundos.

```
Claude Code
  ↓ tools/call: find_symbol_references
RoslynGate MCP
  ↓ valida argumentos
SymbolResolver
  ↓ nombre o file + line
Roslyn Workspace
  ↓ SymbolFinder.FindReferencesAsync
Resultado semántico
  ↓ orden, dedupe, offset, limit
JSON compacto
  ↓ refs: [archivo, línea, tipo]
Agente
```

Ejemplo de respuesta

```
{
  "q": "ViewModels.DatabaseAccess.Doors",
  "definition": {
    "n": "Doors",
    "k": "class",
    "f": "ViewModels/DatabaseAccess/Doors.cs",
    "l": 16
  },
  "total": 167,
  "offset": 0,
  "truncated": true,
  "nextOffset": 200,
  "refs": [
    ["ViewModels/DatabaseAccess/Doors.cs", 22, "read"],
    ["Services.cs", 1820, "read"]
  ]
}
```

Si la página se trunca, el agente recibe `nextOffset` y un `hint`. No tiene que calcular la siguiente página ni adivinar.

04 — REFERENCIAS

Cómo se sabe que una referencia existe

Una referencia no se busca por texto: se pregunta al modelo semántico de Roslyn qué símbolo está en cada nodo del árbol de sintaxis.

1 Resolver el símbolo

Por nombre cualificado o por archivo + línea. Si hay varios candidatos, RoslynGate devuelve candidatos en vez de adivinar.

2 FindReferencesAsync

SymbolFinder recorre la compilación y devuelve ubicaciones que apuntan al mismo símbolo, incluyendo proyectos referenciados.

3 Clasificar

Cada uso se etiqueta como declaración, llamada, herencia, lectura o implícito.

4 Ordenar y paginar

Archivo, línea, columna y tipo con orden estable. El resultado es determinista y acotado.

Casos que el texto no distingue

CASO	QUÉ HACE ROSLYN
Overloads	Distingue la firma y los tipos de parámetros.
Partial classes	Une las partes en un mismo símbolo.
Interfaces	Distingue implementación de llamada concreta.
Código generado	Puede incluir documentos generados por Razor o source generators.
Comentarios / strings	No son referencias semánticas y no se cuentan.

05 — ARQUITECTURA

Tres capas, una dirección

El diseño separa protocolo, consultas y composición. Cada capa tiene una responsabilidad y no se salta a las otras.

Cli

argv, logging, exit codes, `--check`, composición del host. No habla Roslyn directamente.

Mcp

Herramientas MCP, JSON compacto, errores accionables, paginación e instrucciones. No referencia Roslyn.

Engine

Solución, MSBuildWorkspace, SymbolResolver, SymbolFinder, jerarquía y búsqueda. No conoce MCP.

El snapshot

RoslynGate no mantiene un índice en disco. Mantiene un snapshot en memoria de la solución cargada. Las consultas reutilizan compilaciones y símbolos; el proceso MCP vive durante la sesión del agente.

> *La primera consulta paga MSBuild. Las siguientes viven del snapshot. Esa es la diferencia entre “tarda 27 segundos” y “responde en milisegundos”.*

Contrato de herramientas

HERRAMIENTA	USO
`get_symbol_definition`	Resolver un símbolo por nombre o file + line.
`find_symbol_references`	Referencias semánticas paginadas; `includeDefinition` evita una llamada extra.
`get_type_hierarchy`	Base, interfaces, derivados e implementaciones.
`search_symbols`	Buscar por fragmento cuando no se conoce el nombre completo.
`reload_workspace`	Recargar tras cambios de proyecto; el agente lo usa cuando ve `stale`.

Cómo lo usa un agente de IA

El agente no “ejecuta Roslyn”. Llama herramientas MCP. La magia está en convertir una pregunta ambigua en una consulta semántica y devolver solo lo necesario.

1 Definir

El agente pide la definición de un nombre exacto. Si hay ambigüedad, recibe candidatos y elige o pide precisión.

2 Referenciar

Pide usos semánticos. Recibe archivo, línea y tipo de uso, no archivos completos.

3 Paginar

Si `truncated=true`, usa `nextOffset` y repite. Nunca inventa la página siguiente.

4 Responder

Devuelve `file:line` al usuario, con la certeza de que el símbolo fue resuelto por Roslyn.

La skill y las instrucciones

Una skill de Claude Code puede indicar: “para un nombre exacto, llama `find_symbol_references` con `includeDefinition=true`; no llames también a `get_symbol_definition`”. Eso reduce turnos y contexto repetido.

```
---
name: roslyn-symbols
description: Resolve .NET symbols with RoslynGate before Grep.
allowed-tools: mcp__roslyngate__get_symbol_definition mcp__roslyngate__find_symbol_references
---
For an exact name, call find_symbol_references with includeDefinition=true.
If truncated, call again with offset=nextOffset.
```

> La skill no mide el ahorro por sí sola. Para eso hace falta un A/B: mismo prompt con y sin RoslynGate, comparando tokens y coste reales.

07 — RENDIMIENTO

Tokens, latencia y el precio de MSBuild

La herramienta gana cuando evita que el agente lea archivos enteros. El coste es una carga inicial de MSBuild y más turnos si el agente no usa bien las herramientas.

MÉTRICA	CON ROSLYNGATE	SIN ROSLYNGATE	DIFERENCIA
3 símbolos · tokens	94.158	142.506	33,9% menos
3 símbolos · coste	$0,07	$0,10	30% menos
6 símbolos · tokens	103.904	131.865	21,2% menos
6 símbolos · coste	$0,14	$0,18	22% menos
Carga inicial	~27 s	—	una vez
Consultas calientes	4–11 ms	~10 ms	comparable

Cómo leer los números

- Los tokens incluyen lecturas de caché; el coste viene de `total_cost_usd` de Claude Code.
- La latencia con RoslynGate es mayor porque cada proceso nuevo carga el workspace; una sesión interactiva lo conserva.
- En la consulta de 6 símbolos RoslynGate devolvió 167 referencias y el baseline 157: no fue truncamiento.
- Las cifras son una muestra, no una promesa universal.

> El benchmark usa `--no-prewarm` para medir el peor caso. En uso real, el prewarm empieza al abrir la sesión y la primera consulta puede notarse menos.

08 — LÍMITES

Lo que RoslynGate no promete

Un documento técnico honesto incluye lo que la herramienta no hace. Estos límites también explican por qué el diseño es read-only y por qué el SDK sigue siendo necesario.

1 SDK obligatorio

El runtime va incluido, pero MSBuild necesita el SDK .NET. RoslynGate respeta `global.json` y avisa si falta el SDK requerido.

2 Snapshot, no índice

Si un archivo cambia después de cargar, el snapshot puede quedar viejo. La solución es watcher incremental, `stale` y recarga explícita.

3 Sin texto libre

No busca comentarios, strings o documentación por texto. Para eso grep/rg sigue siendo mejor.

4 Sin daemon

Cada proceso MCP carga su workspace. No hay caché en disco ni servidor compartido entre sesiones.

Frescura: ya implementada

El watcher está activo por defecto y actualiza archivos `.cs` en memoria. Los cambios estructurales (`.csproj` , `.razor` , `global.json`) marcan `stale` . La quinta herramienta, `reload_workspace` , recarga sin reiniciar el agente.

```
stale: true
changed: ["ViewModels/DatabaseAccess/Doors.cs"]
try: { tool: "reload_workspace", arguments: {} }
```

Un `0` con `stale: true` no significa “no hay referencias”. La IA debe recargar y reintentar antes de concluir.

09 — ESCALA

Qué se puede construir encima

La base ya separa capas y expone un contrato MCP. Eso permite crecer sin convertir el motor en un monolito.

Watcher incremental

Editar un `.cs` y consultar sin reiniciar. Cambios de proyecto marcan `stale`.

Overloads por tipo

`parameterTypes=["string"]` resuelve la firma sin la vuelta de “elegí por archivo+línea”.

Análisis de impacto

Antes de cambiar una interfaz, listar implementaciones y llamadas afectadas.

CI y revisión de PRs

Usar el mismo motor para verificar referencias y dependencias en pipeline.

Mapas de arquitectura

Grafo de proyectos, capas y dependencias a partir de referencias semánticas.

Múltiples soluciones

Filtros por proyecto o solución para monorepos, con carga selectiva.

Lo que no haría

No convertiría RoslynGate en un buscador de texto libre ni en un editor. Eso rompe la propuesta: precisión semántica, read-only y bajo contexto. Para texto, grep; para símbolos, Roslyn.

> *La herramienta no intenta ser un IDE. Intenta darle a un agente la respuesta semántica mínima que necesita para no leer medio repositorio.*

READ-ONLY · SEMÁNTICO · MCP · DETERMINISTA · PAGINADO · SDK RESPETADO